

CMSC411: Computer Systems Architecture
Project 2: Tomasulo's Algorithm
Due: 04/15/2024, 11:59 pm, EST (via Gradescope)

Important policies

1. Sharing of code between students is viewed as cheating and will receive appropriate action in accordance with university policy.
2. It is acceptable for you to compare your results, and only your results, with other students to help debug your program. It is not acceptable to collaborate either on the code development or on the final experiments.
3. You should do all your work in the C or C++ programming language and should be written according to the C99 or C++11 standards, using only the standard libraries.

Project Description

In this project, we will be designing a simulator that implements a simplified version of Tomasulo's algorithm. You are asked to design the register alias table (RAT) and reservation stations (REST).

Each line of the trace file (e.g., MEM 1 2 3) is in the following format: **OP RD RS RT**.

- **OP** indicates the operation type. There will be different types of instructions, each with a different latency (e.g., ADD takes 2 cycles, DIV takes 15 cycles, MEM takes 20).
- **RD** is the destination register number (e.g., 1 for R1)
- **RS** is the first source register number. If there is none (for example, an immediate value), the "number" is -1.
- **RT** is the second source register number.

You will be provided driver code that will convert the traces to instructions. You are only responsible for filling in the following functions (along with any other classes and variables as needed). FUs are pipelined. You start from cycle number 1.

Specification of Simulator

Before going to the details of what you need to do, we briefly explain a general overview of the code. For this project, we provide you with following files and folders:

- [schedulersim.cpp](#): You need to make your changes in this file. Follow these twelve numbers in the code to see the parts you need to complete: 1, 2.1, 2.2, 3.1, 3.2, 4.1, 4.2, 4.3, 4.4, 5.1, 5.2, 5.3.
- [schedulersim_driver.cpp](#): This file includes the main() function and runs the simulator for you. You don't have to change anything in this file. But feel free to read it to better get familiar with the code.
- [schedulersim.hpp](#): This file includes the definition of the functions, structs, etc. You don't have to change anything in this file. But feel free to read it to better get familiar with the code.
- [Makefile](#): This is the makefile that you need to run to compile your code:

- \$make
- \$make clean
- /test-traces and /test-output: These two folders respectively include a group of test traces that your simulator will read and the output that it must generate if your simulator works correctly.
- /traces: This folder includes real-world benchmarks that you will use to optimize your code for.
- validate-test-traces.sh: You will be running this script to test your code. Basically, this script runs schedulersim for all the traces in /test-traces, writes the outputs in /myoutput, compares everything in /test-output with /myoutput and prints out the differences.

We have provided you with an implementation of REST and RAT. Some functions (e.g., `add_entry`, `init_table`, `is_empty`, `fire_ready`, `count_active`, `complete_insts`, `complete_insts`, `free_table`) in REST, however, are not completed. You may either choose to complete these functions or redefine your brand new REST and RAT in a way you want.

Explanation of functions you need to fill in:

1. `void scheduler_per_fu_init(int num_registers, int rs1, int rs2, int rs3)`
 - This function is called if the type of scheduler uses per-FU reservation stations.
 - `void scheduler_unified_init(int num_registers, int rs_size)` is provided as an example, which is called if the type of scheduler uses a unified reservation station.
2. `bool scheduler_try_issue(op_type op, int dest, int src1, int src2)`
 - To complete this function, you may first complete `add_entry`
 - This function tries to issue a new instruction with the given arguments. If successful, returns true, if not (i.e., the RS is full), return false.
3. `bool scheduler_completed()`
 - Return true if all instructions are completed and cleared.
 - To complete this function, you may first complete `is_empty`
4. `void scheduler_start_ready()`
 - To complete this function, you may first complete `fire_ready`
 - Starts any ready instructions
5. `bool scheduler_clear_completed()`
 - To complete this function, you may first complete `complete_insts`
 - Clears any completed instructions.

Statistics (output)

The simulator outputs the following statistics after completion of the run:

1. Number of instructions
2. Number of cycles
3. Instructions Per Cycle (IPC)

4. Number of times the issue was stalled (issue failed)
5. Maximum number of instructions started at once
6. Maximum number of instructions completed (wrote back) at once
7. Maximum number of instructions active per FU

Validation

Several test traces will be provided in /test-traces directory along with their correct output in /output directory. You must run your simulator and debug it until it matches 100% all the statistics in the validation outputs posted on the website. It is highly recommended that you first try to work through each trace for each scheduler type by hand.

Experiments

For each benchmark in the traces directory, vary any input parameters (e.g., RS type, entries per RS) to design an instruction scheduler subject to the following goals:

1. You have a total budget of 10 entries for the reservation stations (10 all in the unified RS, or X for each RS that sums up to 10 entries)
2. The scheduler should have the highest possible IPC at the end. You may vary any parameter

Deliverables

What to hand in via Gradescope:

- **schedulersim.cpp**
 - The commented source code for the simulator program itself.
- **<UID>_prj2.pdf**
 - A document with the design results of the experiments for each trace file, with a persuasive argument of the choices that were made. (An argument may be as simple as an explanation of the search procedure used to find the designs and a statement about why the procedure is complete.) This argument should include output from runs of your program. (There are multiple answers for each trace file, so I will know which students have "collaborated" inappropriately!)

Remember that your code must compile and run on a current variant of Linux (i.e., Debian, Red Hat, Ubuntu) running on an x86 architecture (i.e., Intel or AMD).

Grading Rubric

0% You do not hand in anything

Up to +36% Your simulator doesn't run, does not work, but you hand in significant commented code (depending on how many parts of the code you completed you gain maximum 36%, 3% per each)

+48% Your simulator matches the validation outputs (8% per output)

+16% You ran all experiments and found the best-performing schedulers

Important note: Late submissions will be deducted 25% per day